

O Engenheiro de DevEx e o Desenvolvedor "Preguiçoso" (Focado em Eficiência)

Transcrição da prática 1 · 1 de set. de 2026 14:53

Felipe Gouveia 00:00

Olá! Olha, eu vi que você está animado com esse Claude Code. Eu entendo que ele ajuda a editar arquivos, mas para mim, ferramenta de terminal boa é ferramenta que segue processo. Eu não quero o time 'conversando' com a IA o dia todo; eu quero que eles executem tarefas.

Me explica o básico: O que são esses comandos slash e por que eles seriam melhores do que eu apenas digitar o que eu quero em português? E essa história de 'comandos customizados'... eu consigo criar um comando que force o CI

00:52 Você

estou aprendendo por vafor me explique melhor

Felipe Gouveia 00:57

Claro! Os comandos slash, como '/bug' ou '/test', são atalhos inteligentes que transformam tarefas complexas em uma única ação no terminal. Por exemplo, ao invés de você rodar vários comandos para iniciar testes de uma aplicação, você só precisa digitar '/test', e ele cuida de todo o processo. Além disso, com os comandos customizados, você pode ensinar o Claude a seguir exatamente as regras e formatos que a sua equipe usa, como criar pull requests com padrões específicos. Isso significa mais agilidade e menos chances de erro! Quer um exemplo prático?

01:33 Você

sim, por favor

Felipe Gouveia 01:36

Beleza! Imagina que você precisa abrir um pull request. Normalmente, você teria que criar uma nova branch, escrever uma mensagem de commit no padrão da empresa, ir para o GitHub e preencher os detalhes do PR. Com um comando customizado no Claude Code, você só digita algo como '/pr adiciona funcionalidade X', e ele faz tudo automaticamente: cria a branch, escreve a mensagem no formato correto e até abre o PR já preenchido. Show, né? Isso economiza um tempão. O que acha dessa ideia?

02:36 Você

muito boa, mas alguma coisa para falar?

Felipe Gouveia 02:40

Além disso, tem uma coisa bem legal: esses comandos não são só atalhos burros. Eles entendem o contexto! Por exemplo, se você usa o comando `/review`, ele pode identificar automaticamente quais arquivos mudaram, sugerir correções ou rodar lints antes de enviar para revisão. Isso elimina passos manuais e te deixa focar no que importa: o código. Quer saber mais sobre como isso funciona?

03:09 Você

sim por favor.

Felipe Gouveia 03:12

Legal! Então, quando você usa um comando baseado em contexto, tipo `/review`, o Claude Code analisa o estado atual do código, como mudanças locais, conflitos pendentes ou até dependências. A partir disso, ele toma decisões inteligentes, como rodar apenas os lints necessários, sugerir reviewers com base no histórico ou até pré-formatar a mensagem de revisão. É como se fosse um colega de time super antenado que já sabe o que fazer antes mesmo de você explicar! Dá pra imaginar o quanto isso acelera o fluxo, né?

03:45 Você

muito bom

Felipe Gouveia 03:49

Que bom que você gostou! Essa é a ideia: transformar o terminal em um verdadeiro assistente que entende o contexto e segue as regras da equipe. Se você tiver mais dúvidas ou quiser explorar outra funcionalidade, é só falar!

04:06 Você

ok

Felipe Gouveia 04:08

Valeu pela conversa! Espero que o Claude Code realmente te ajude a turbinar a produtividade da equipe. Qualquer coisa, só chamar. Até mais!

FIM DA SESSÃO

Esta transcrição inclui conteúdo gerado por IA e é fornecida para seus propósitos de aprendizado pessoal.